



BÀI TẬP LỚN

Đề tài: Xây dựng hệ thống giám sát chất lượng không khí trong nhà máy

Môn học: Xây dựng hệ thống nhúng
Số thứ tự nhóm: 8

Ngô Văn Bộ	MSSV: B22DCCN085
Nguyễn Đức Cường	MSSV: B22DCCN097
Phạm Hải Dương	MSSV: B22DCCN169
Hoàng Văn Nam	MSSV: B22DCCN553
Tổng Duy Nam	MSSV: B22DCCN565

Giảng viên hướng dẫn: TS. Phạm Hoàng Việt

HÀ NỘI, 06/2026

TÓM TẮT NỘI DUNG DỰ ÁN

Dự án xây dựng hệ thống giám sát và xử lý chất lượng không khí trong nhà máy theo thời gian thực sử dụng vi điều khiển ESP32, cảm biến khí MQ-135, module ADC ADS1115 và nền tảng giám sát từ xa qua MQTT. Bài toán xuất phát từ nhu cầu phát hiện sớm rò rỉ hoặc gia tăng nồng độ khí độc trong môi trường sản xuất để kịp thời cảnh báo và xử lý, giảm rủi ro mất an toàn lao động.

Nội dung chính của dự án tập trung vào các khối chức năng cốt lõi gồm thu thập dữ liệu nồng độ khí từ hai khu vực giám sát, xử lý dữ liệu tại thiết bị biên, truyền thông lên HiveMQ Broker và hiển thị trạng thái vận hành trên dashboard. Hệ thống hỗ trợ hai chế độ điều khiển AUTO và MANUAL, đồng thời bổ sung Emergency Mode bằng 3 nút bấm vật lý để can thiệp khẩn cấp tại hiện trường. Về phần cứng, hệ thống sử dụng thêm module ADC ADS1115 16-bit để tăng độ chính xác khi đọc tín hiệu analog, servo SG90 để điều khiển góc mở cửa sổ thông gió, còi SFM-27 để cảnh báo âm thanh, cùng nguồn dự phòng pin lithium 18650, mạch chuyển nguồn YX850 và module LM2596 nhằm duy trì hoạt động liên tục khi mất điện lưới.

Kết quả thử nghiệm cho thấy hệ thống phản hồi nhanh khi nồng độ khí tăng cao, dữ liệu cập nhật liên tục với độ trễ thấp và khả năng điều khiển từ xa ổn định. Hệ thống đã triển khai cập nhật firmware từ xa (OTA), cơ chế Watchdog Timer và khôi phục trạng thái sau reset để tăng độ tin cậy vận hành. Trên cơ sở đó, báo cáo đánh giá mức độ đáp ứng các yêu cầu chức năng và phi chức năng, đồng thời đề xuất các hướng phát triển như tăng cường bảo mật TLS đầy đủ và bổ sung mô-đun AI tóm tắt xu hướng ô nhiễm.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	4
1.1 Lý do và động lực.....	4
1.2 Khảo sát công nghệ.....	4
1.3 Lựa chọn kỹ thuật.....	4
CHƯƠNG 2. NỘI DUNG CHÍNH.....	6
2.1 Xác định bài toán và yêu cầu hệ thống	6
2.1.1 Mô tả bài toán thực tế	6
2.1.2 Yêu cầu chức năng.....	6
2.1.3 Yêu cầu phi chức năng	7
2.1.4 Phân tích ràng buộc.....	7
2.2 Phần cứng của hệ thống	8
2.2.1 Block diagram	8
2.2.2 Vi điều khiển ESP32	8
2.2.3 MQ-135	9
2.2.4 Quạt tản nhiệt 5V mini 4x4	11
2.2.5 Module LCD I2C 16x2.....	11
2.2.6 Module Relay 2 kênh 5V.....	12
2.2.7 Servo SG90	14
2.2.8 Còi SFM-27.....	15
2.2.9 Module ADC ADS1115 16-bit 4 kênh	16
2.2.10 Module YX850 mạch tự động chuyển nguồn dự phòng 5-48VDC	17
2.2.11 Module hạ áp LM2596.....	18
2.2.12 Adapter 5V 2A	19
2.2.13 Nguồn dự phòng pin lithium 18650	20
2.2.14 Bảng cấu hình chân GPIO	21
2.3 Phần mềm của hệ thống	22
2.3.1 Tổng quan chức năng phần mềm.....	22
2.3.2 Phân ngưỡng chất lượng không khí (AQI Classification)	23
2.3.3 Bộ lọc EWMA (Exponential Weighted Moving Average) và Phát hiện lỗi cảm biến.....	23

2.3.4 Cơ chế chống nhiễu I2C khi Relay đóng/ngắt.....	24
2.3.5 Đồng bộ thời gian NTP.....	24
2.3.6 Giao thức truyền tin MQTT.....	25
2.3.7 Thiết kế không sử dụng RTOS (mô hình đối chiếu)	26
2.3.8 Thiết kế có sử dụng RTOS (FreeRTOS trên ESP32)	28
2.3.9 Cơ chế điều khiển từ người dùng	30
2.3.10 Hệ thống khẩn cấp (Emergency Mode)	31
2.3.11 Watchdog Timer và khôi phục trạng thái	31
2.3.12 Cơ chế cập nhật firmware OTA	31
2.4 Phương pháp thực hiện và kết quả	31
2.4.1 Phương pháp thực hiện	31
2.4.2 Kết quả đạt được.....	33
2.4.3 Đánh giá và nhận xét	35
CHƯƠNG 3. KẾT LUẬN.....	36
3.1 Kết luận	36
3.2 Hướng phát triển.....	36
TÀI LIỆU THAM KHẢO	37

DANH MỤC HÌNH VẼ

Hình 2.1	Block diagram của hệ thống	8
Hình 2.2	Bộ mạch ESP32 DevKit V1	8
Hình 2.3	Cảm biến khí MQ-135	10
Hình 2.4	Sơ đồ chân và giao tiếp module cảm biến khí	10
Hình 2.5	Quạt tản nhiệt mini 5V kích thước 4x4	11
Hình 2.6	Module LCD I2C 16x2 PCF8574	12
Hình 2.7	Màn hình LCD đang hoạt động hiển thị nồng độ khí và trạng thái quạt	12
Hình 2.8	Module Relay 2 kênh 5V với sơ đồ chân kết nối	13
Hình 2.9	Servo SG90 (Micro Servo)	14
Hình 2.10	Còi SFM-27 (Active Buzzer)	15
Hình 2.11	Module ADC ADS1115 16-bit 4 kênh (I2C)	16
Hình 2.12	Module YX850 thực tế	18
Hình 2.13	Nguyên lý chuyển nguồn chính và nguồn dự phòng của YX850	18
Hình 2.14	Module hạ áp LM2596	19
Hình 2.15	Adapter nguồn 5V 2A	20
Hình 2.16	Pin lithium-ion 18650 dùng làm nguồn dự phòng	21
Hình 2.17	Hệ thống giám sát chất lượng không khí trong nhà máy	34
Hình 2.18	Giao diện website giám sát và điều khiển hệ thống	34

DANH MỤC BẢNG BIỂU

Bảng 2.1	Cấu hình chân GPIO của ESP32 trong hệ thống	22
Bảng 2.2	Bảng phân ngưỡng chất lượng không khí (AQI) và hành động điều khiển	23
Bảng 2.3	Cấu trúc Topic MQTT	25
Bảng 2.4	Phân chia tác vụ logic khi không sử dụng RTOS	26
Bảng 2.5	Nguyên tắc điều khiển Auto và Manual	27
Bảng 2.6	Phân chia tác vụ khi sử dụng RTOS trên ESP32	29

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
ESP32	Vi điều khiển tích hợp Wi-Fi và Bluetooth
ADC	Bộ chuyển đổi tương tự sang số
I2C	Chuẩn giao tiếp hai dây cho cảm biến và ngoại vi
MQTT	Giao thức nhắn tin nhẹ dùng cho IoT
GPIO	Chân vào ra mục đích chung

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Lý do và động lực

Trong môi trường công nghiệp, kiểm soát chất lượng không khí là yêu cầu tiên quyết để bảo vệ sức khỏe người lao động và đảm bảo an toàn vận hành. Các công đoạn sản xuất thường phát sinh các khí độc hại, khí dễ cháy như NH_3 , CO , hoặc các hợp chất hữu cơ bay hơi (VOCs). Nếu không có hệ thống giám sát liên tục, các tác nhân này dễ dẫn đến nguy cơ ngộ độc hoặc cháy nổ nghiêm trọng.

Hiện nay, các hệ thống công nghiệp (PLC/SCADA) thường rất đắt đỏ và khó triển khai cho các nhà máy quy mô vừa và nhỏ. Ngược lại, việc kiểm tra thủ công lại không đảm bảo tính tức thời. Nhận thấy tiềm năng của công nghệ IoT, nhóm quyết định thực hiện đề tài *Hệ thống giám sát chất lượng không khí trong nhà máy* nhằm:

- **Giám sát 24/7:** Cập nhật giá trị khí theo thời gian thực cho hai khu vực đo.
- **Phản ứng tức thời:** Tự động kích hoạt quạt, còi báo động và cập nhật trạng thái khi vượt ngưỡng.
- **Số hóa quản lý:** Giám sát từ xa qua Dashboard, lưu trữ dữ liệu trên Cloud và hướng tới phân tích thông minh bằng AI.

1.2 Khảo sát công nghệ

Thị trường hiện có hai hướng tiếp cận chính:

1. **Hệ thống công nghiệp chuyên dụng:** Sử dụng PLC kết hợp cảm biến độ chính xác cao. Ưu điểm là cực kỳ ổn định nhưng chi phí đầu tư và bảo trì rất lớn.
2. **Hệ thống IoT mã nguồn mở:** Sử dụng các dòng vi điều khiển mạnh mẽ như ESP32. Hướng đi này đang trở thành xu hướng nhờ chi phí thấp, tính linh hoạt cao và khả năng kết nối không dây mạnh mẽ.

Trong bài tập lớn này, nhóm tập trung vào giải pháp IoT kết hợp Cloud. Việc đưa dữ liệu lên đám mây không chỉ giúp giám sát từ xa mà còn mở ra khả năng phân tích xu hướng môi trường, một yếu tố mà các hệ thống truyền thống tại chỗ thường thiếu sót.

1.3 Lựa chọn kỹ thuật

Hệ thống được thiết kế theo kiến trúc Edge-to-Cloud, tối ưu hóa hiệu suất giữa thiết bị biên và nền tảng điện toán đám mây:

- **Kiến trúc hệ thống:** Khối phần cứng gồm ESP32, 02 cảm biến MQ-135, ADS1115, relay, LCD, servo, buzzer, nút khẩn cấp và mạch nguồn dự phòng; khối phần mềm chạy FreeRTOS để xử lý sensor, control, LCD, MQTT và OTA; khối Cloud gồm HiveMQ Broker, Dashboard và NTP; khối nguồn gồm adapter, YX850, LM2596 và pin 18650.
- **Vi điều khiển:** Nhóm chọn ESP32 làm bộ não trung tâm. Với lợi thế xử lý đa nhiệm, tích hợp WiFi và hỗ trợ tốt các thư viện IoT, ESP32 giúp xử lý mượt mà đồng thời việc đọc cảm biến, điều khiển chấp hành và truyền dữ liệu mạng.
- **Cảm biến:** Sử dụng 02 cảm biến MQ-135 qua ADS1115 tại các khu vực trọng yếu. Dữ liệu được đọc dưới dạng raw ADC để đối chiếu ngưỡng và hiển thị trạng thái chính xác hơn.
- **Giao tiếp:** Giao thức MQTT qua HiveMQ Broker được lựa chọn nhờ tính nhẹ, độ trễ thấp và hỗ trợ tốt kết nối an toàn. Hệ thống vận hành song song các chế độ AUTO, MANUAL và EMERGENCY qua Dashboard hoặc nút bấm tại chỗ.
- **Quản lý thông minh:** Hệ thống hỗ trợ cập nhật firmware từ xa (OTA) và đồng bộ thời gian NTP, cho

phép hiệu chỉnh logic điều khiển mà không cần tháo lắp thiết bị.

- **Độ tin cậy nguồn điện:** Điểm nhấn kỹ thuật của nhóm là mạch nguồn dự phòng dùng pin 18650, module YX850 và LM2596, bảo đảm việc giám sát không bị gián đoạn ngay cả khi nhà máy xảy ra sự cố mất điện.

CHƯƠNG 2. NỘI DUNG CHÍNH

2.1 Xác định bài toán và yêu cầu hệ thống

2.1.1 Mô tả bài toán thực tế

Bối cảnh triển khai của hệ thống là các môi trường công nghiệp như xưởng sản xuất hóa chất, khu vực sơn phủ hoặc các phòng chứa dung môi, nơi rò rỉ khí độc như Ammonia, Benzene hoặc VOC có thể gây ngộ độc cấp tính và cháy nổ. Việc giám sát thủ công thường không bảo đảm độ phủ và tính liên tục, dẫn đến phản ứng chậm khi nồng độ khí vượt ngưỡng an toàn.

Giải pháp được đề xuất là xây dựng một hệ thống giám sát và xử lý khí độc thông minh sử dụng vi điều khiển ESP32, 02 cảm biến MQ-135 qua ADS1115, LCD hiển thị cục bộ và khối chấp hành gồm quạt, còi báo động và servo. Hệ thống không chỉ quan trắc nồng độ khí theo thời gian thực mà còn tự động phản hồi bằng cách kích hoạt các cơ cấu chấp hành khi phát hiện ô nhiễm. Dữ liệu được quản lý tập trung qua Cloud để phân tích xu hướng và phục vụ báo cáo an toàn lao động, đồng thời hệ thống có nguồn dự phòng để duy trì hoạt động khi mất điện.

2.1.2 Yêu cầu chức năng

Hệ thống tập trung vào khả năng phát hiện khí độc và phản ứng nhanh.

a, Thu thập dữ liệu nồng độ khí

- Phát hiện và đo lường nồng độ các loại khí độc hại như Amoniac, Sulfide, hơi Benzen và VOC bằng cảm biến chất lượng không khí MQ135.
- Đọc tín hiệu analog qua ADS1115 và quy đổi về giá trị raw ADC 12-bit để xử lý ngưỡng.

b, Xử lý và điều khiển thiết bị (Actuation Control)

- **Chế độ Tự động (AUTO):** Hệ thống tự động kích hoạt quạt thông gió, còi báo động và các cơ cấu hỗ trợ như servo mở cửa sổ ngay khi nồng độ khí đo được vượt ngưỡng cài đặt an toàn.
- **Chế độ Thủ công (MANUAL):** Người quản lý có thể điều khiển cưỡng bức trạng thái quạt, còi và cửa sổ trực tiếp từ Dashboard thông qua giao thức MQTT.
- **Chế độ Khẩn cấp (EMERGENCY):** Các nút bấm tại hiện trường cho phép kích hoạt trạng thái khẩn cấp với ưu tiên cao nhất để xử lý nhanh khi có sự cố.

c, Truyền thông và giám sát

- Gửi gói tin dữ liệu định dạng JSON lên HiveMQ Broker thông qua giao thức MQTT.
- Hiển thị trạng thái quạt và biểu đồ nồng độ khí trên Web Dashboard thời gian thực.
- Hiển thị nhanh trạng thái các phòng trên LCD I2C 16x2 ngay tại thiết bị.

d, Quản trị nâng cao

- **Cập nhật OTA:** Hỗ trợ cập nhật firmware từ xa cho ESP32 để hiệu chỉnh cảm biến hoặc thay đổi logic điều khiển mà không cần can thiệp phần cứng.
- **Đồng bộ thời gian NTP:** Đồng bộ timestamp để ghi log và lưu lịch sử telemetry chính xác.
- **AI Summary (định hướng):** Chức năng tóm tắt xu hướng ô nhiễm theo giờ bằng AI đã được đề xuất nhưng chưa triển khai trong phiên bản hiện tại.

2.1.3 Yêu cầu phi chức năng

a, Tính thời gian thực (Real-time)

- Độ trễ dữ liệu từ cảm biến MQ135 đến Dashboard không quá 3 giây.
- Thời gian phản hồi kích hoạt quạt khi phát hiện rò rỉ khí phải dưới 5 giây.

b, Quản lý năng lượng (Power Management)

- Sử dụng nguồn chính từ Adapter 5V-2A.
- Tích hợp hệ thống pin dự phòng Lithium 18650 và mạch chuyển nguồn tự động YX850 (UPS) để duy trì hoạt động giám sát khi mất điện lưới.
- Áp dụng chế độ Sleep Mode giữa các chu kỳ truyền tin để tối ưu điện năng khi chạy pin.

c, Độ tin cậy và bảo mật

- Đảm bảo tỷ lệ truyền gói tin thành công đạt ít nhất 98% trong môi trường công nghiệp.
- Xác thực thiết bị và người dùng qua mã thông báo JWT; mã hóa dữ liệu truyền tải qua TLS/SSL.

d, Chi phí (Cost)

Tổng chi phí đầu tư phần cứng nếu triển khai thực tế với số lượng lớn, ví dụ 20 trạm cảm biến, phải dưới 250 triệu VNĐ.

2.1.4 Phân tích ràng buộc

a, Tài nguyên thiết bị (Hardware Limits)

- CPU/RAM: ESP32 có dung lượng SRAM giới hạn (520 KB), các tác vụ phân tích dữ liệu lớn hoặc AI phải được đẩy lên Cloud xử lý.
- Lập trình đa nhiệm: Sử dụng FreeRTOS để phân chia các Task (taskEmergency, taskOTA, taskControl, taskSensor, taskLCD, taskMQTT) với mức ưu tiên khác nhau, tránh xung đột khi điều khiển quạt.

b, Ràng buộc cảm biến

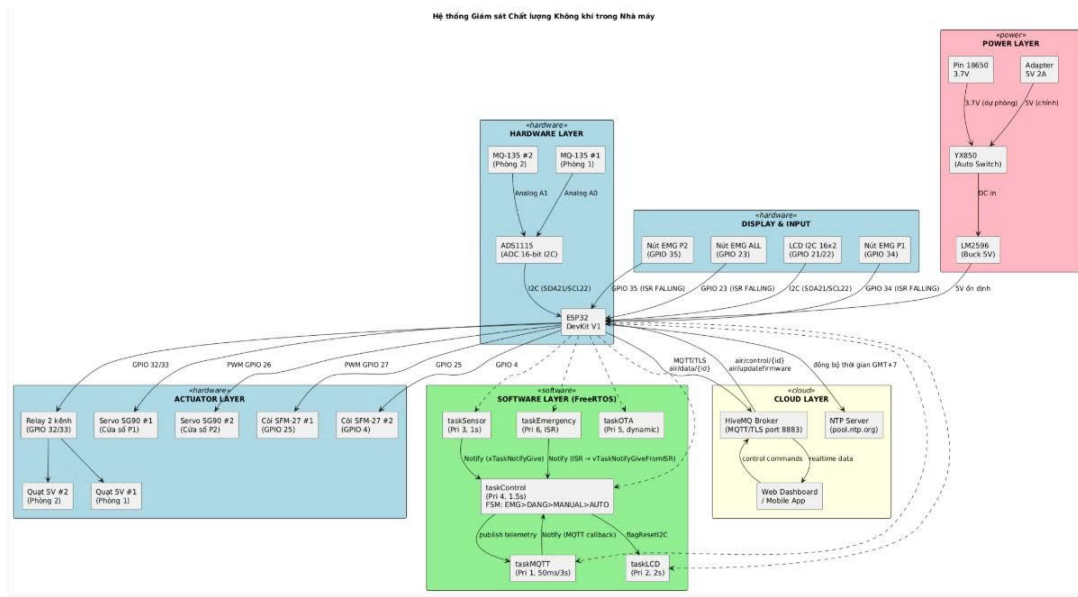
- Cảm biến MQ135 cần thời gian sấy (Warm-up) để bộ phận nhiệt đạt đủ nhiệt độ đo chính xác, yêu cầu nguồn cấp ổn định 5V.

c, Môi trường hoạt động

- Thiết bị phải vận hành ổn định trong dải nhiệt độ công nghiệp từ 0°C đến 70°C.
- Vỏ hộp thiết bị đạt tiêu chuẩn IP65 để bảo vệ mạch điện khỏi hơi dung môi và bụi bẩn trong nhà máy.

2.2 Phần cứng của hệ thống

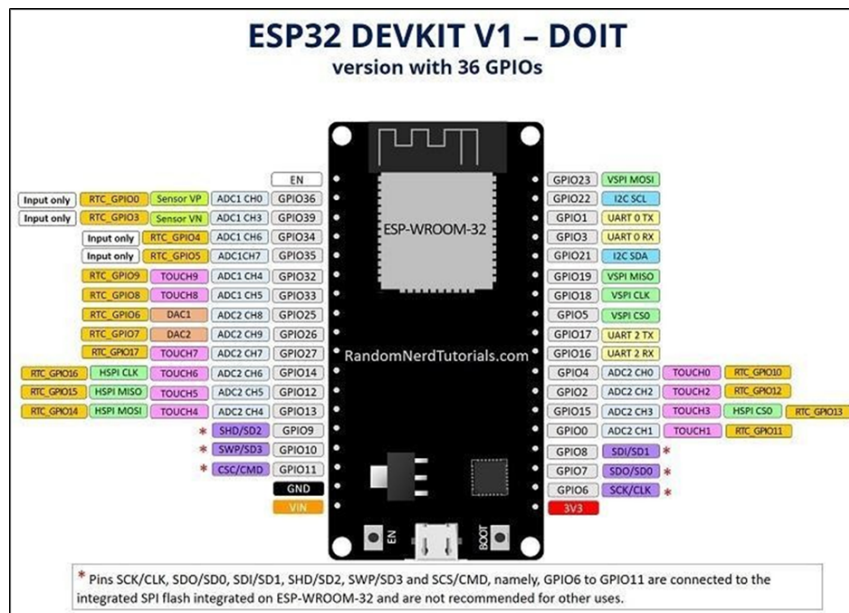
2.2.1 Block diagram



Hình 2.1: Block diagram của hệ thống

2.2.2 Vi điều khiển ESP32

ESP32-WROOM-32D là một module Wi-Fi + Bluetooth tích hợp, phát triển bởi Espressif Systems, được sử dụng rộng rãi trong IoT (Internet of Things), tự động hóa và các thiết bị nhúng thông minh nhờ khả năng xử lý mạnh mẽ và kết nối không dây. Trên thực tế, người ta thường sử dụng bo mạch phát triển ESP32 DevKit V1 tích hợp sẵn module ESP32-WROOM-32D để dễ dàng lập trình và kết nối.



Hình 2.2: Bo mạch ESP32 DevKit V1

ESP32 được lựa chọn thay vì Arduino vì các lý do sau:

- Tích hợp sẵn WiFi, phù hợp hệ thống IoT sử dụng MQTT.
- Hiệu năng cao, đáp ứng xử lý dữ liệu và truyền thông đồng thời.

- Bộ nhớ lớn, hỗ trợ tốt các giao thức mạng.
- Giảm độ phức tạp phần cứng (không cần module WiFi ngoài).
- Dễ mở rộng và phát triển trong tương lai.

Trong khi đó, Arduino không phù hợp do:

- Không có WiFi tích hợp.
- Hiệu năng thấp.
- Hạn chế bộ nhớ.
- Khó triển khai hệ thống IoT hoàn chỉnh.

Các thông số chính:

- CPU: Xtensa dual-core 32-bit LX6 (tốc độ tối đa 240 MHz).
- Bộ nhớ: 448 KB ROM, 520 KB SRAM, 4 MB Flash (trên module).
- Kết nối: Wi-Fi 802.11 b/g/n, Bluetooth v4.2 (Classic + BLE).
- GPIO: 34 chân I/O, hỗ trợ PWM, ADC (12-bit), DAC (8-bit), SPI, I2C, UART.
- Điện áp hoạt động: 3.0V-3.6V (DevKit hỗ trợ cấp từ 5V qua cổng USB).

Các linh kiện chính trên bo mạch ESP32 DevKit 32D:

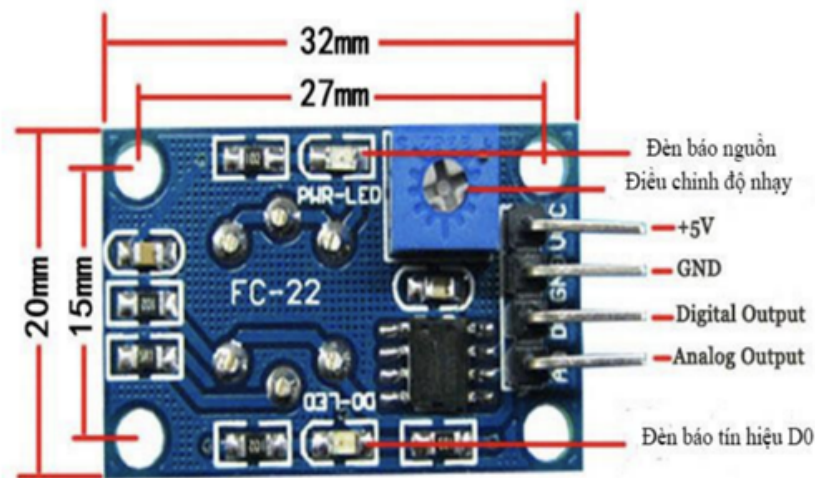
- **Module ESP32-WROOM-32D:**
 - Thành phần trung tâm (SoC ESP32 + Flash 4 MB).
 - Chứa bộ xử lý, bộ nhớ, WiFi/Bluetooth và mạch RF.
- **USB-to-UART Bridge (CP2102 hoặc CH340):**
 - Cho phép nạp chương trình và giao tiếp với máy tính thông qua cổng USB.
 - Chuyển đổi tín hiệu USB sang UART.
- **Ổn áp AMS1117-3.3V:** Giúp chuyển đổi điện áp từ 5V (USB) hoặc Vin xuống 3.3V để nuôi ESP32.
- **Cổng Micro-USB:** Dùng để cấp nguồn (5V) và nạp code.
- **Nút nhấn:**
 - EN (Reset): Reset lại vi điều khiển.
 - BOOT (IO0): Giữ để đưa ESP32 vào chế độ nạp chương trình.
- **Thạch anh (Crystal 40 MHz):** Tạo xung clock ổn định cho chip ESP32.
- **LED chỉ thị (thường nối với GPIO2):** Báo hiệu nguồn hoặc được dùng test xuất tín hiệu.
- **Hàng chân header (Male header pins):** Xuất các chân GPIO, nguồn (3.3V, 5V), GND để dễ dàng kết nối với breadboard hoặc module khác.
- **Các tụ điện, điện trở dán (SMD):** Dùng để lọc nhiễu, ổn định nguồn, kéo lên hoặc kéo xuống cho các chân tín hiệu.

2.2.3 MQ-135

MQ-135 là cảm biến khí bán dẫn sử dụng vật liệu thiếc dioxide. Nó được thiết kế đặc biệt để phát hiện nhiều loại khí độc hại và khí gây ô nhiễm môi trường. Do độ nhạy cao và giá thành rẻ, nó là lựa chọn hàng đầu cho các ứng dụng Air Quality Monitor (Giám sát chất lượng không khí) trong nhà và văn phòng.



Hình 2.3: Cảm biến khí MQ-135



Hình 2.4: Sơ đồ chân và giao tiếp module cảm biến khí

Các loại khí phát hiện (Detection Range):

- Amoniac: Rất nhạy.
- Sulfide và hơi Benzen: Nhạy.
- Hơi rượu (Alcohol), khói (Smoke): Nhạy trung bình.
- Các oxit nitơ: Phát hiện được.
- CO₂: Có khả năng phát hiện nhưng không chuyên dụng (thường dùng để ước lượng mức độ ô nhiễm không khí nói chung).

Thông số kỹ thuật (Technical Specifications):

- Điện áp hoạt động (VCC): 5V DC $\pm$ 0.1V (lưu ý: ESP32 dùng logic 3.3V nhưng chân VCC của cảm biến phải nối 5V để bộ phận sấy hoạt động đủ nhiệt).
- Điện áp mạch sấy (Heater Voltage): 5V DC/AC.
- Công suất tiêu thụ: Khoảng 800mW (cảm biến sẽ khá nóng khi hoạt động, đây là bình thường).
- Đầu ra dữ liệu:
 - Analog (A0): Điện áp biến thiên từ 0 - 5V (tương ứng với nồng độ khí).
 - Digital (D0): Tín hiệu 0 hoặc 1 (được so sánh qua Op-Amp LM393 tích hợp trên module, chỉnh

ngưỡng bằng biến trở).

- Phạm vi phát hiện: 10ppm - 1000ppm (tùy loại khí).

2.2.4 Quạt tản nhiệt 5V mini 4x4

Quạt tản nhiệt nhỏ gọn, sử dụng nguồn 5V phù hợp cho hệ thống.



Hình 2.5: Quạt tản nhiệt mini 5V kích thước 4x4

2.2.5 Module LCD I2C 16x2

Module LCD I2C 16x2 là một màn hình ký tự 16 cột $\times$ 2 hàng kết hợp với mô-đun chuyển đổi I2C PCF8574. Thiết bị này cho phép hiển thị thông tin tại chỗ (offline display) mà không cần phụ thuộc vào kết nối internet hoặc điện thoại di động.

Trong hệ thống này, LCD phục vụ mục đích quan trọng:

- Hiển thị thông tin nồng độ khí đo được từ cảm biến MQ-135 (giá trị raw ADC).
- Hiển thị mức độ chất lượng không khí (GOOD, MOD, BAD, DANG).
- Hiển thị trạng thái quạt (ON/OFF) và chế độ hoạt động (AUTO/MANUAL) của từng phòng.
- Giúp vận hành viên hoặc kỹ sư bảo trì đứng tại trạm có thể kiểm tra tình trạng hệ thống ngay lập tức mà không cần mở điện thoại hay laptop.

Thông số kỹ thuật:

- Loại màn hình: LCD ký tự 16 $\times$ 2 (16 cột, 2 hàng).
- Giao diện: I2C (2 dây: SDA, SCL), sử dụng chip PCF8574 làm bộ chuyển đổi.
- Điều chỉnh độ sáng: Có biến trở trên module.
- Điện áp hoạt động: 5V DC.
- Dòng điện: Khoảng 50-100 mA (tùy độ sáng).

Cấu hình trong dự án:

- Địa chỉ I2C mặc định: 0x27 (có thể khác tùy module).
- Chân SDA: GPIO 21 của ESP32.
- Chân SCL: GPIO 22 của ESP32.
- Thư viện sử dụng: LiquidCrystal_I2C.

Hoạt động:

- Trong task LCD, hệ thống xoay vòng (paging) hiển thị thông tin của hai phòng mỗi 2 giây.
- Mỗi trang hiển thị khoảng 2 dòng với các thông tin: nồng độ khí, mức cảnh báo, trạng thái quạt, chế độ điều khiển.
- Khi có sự kiện chuyển trạng thái quạt (relay đóng/ngắt), task LCD sẽ re-initialize I2C và LCD để chống lại nhiễu đường bus I2C gây bởi dòng cảm ứng ngược từ cuộn relay.



Hình 2.6: Module LCD I2C 16x2 PCF8574



Hình 2.7: Màn hình LCD đang hoạt động hiển thị nồng độ khí và trạng thái quạt

2.2.6 Module Relay 2 kênh 5V

Module Relay 2 kênh là thiết bị chuyển mạch điện tử dùng để điều khiển các tải điện (như quạt, mô-tơ, đèn) hoạt động ở mức công suất cao từ các tín hiệu điều khiển mức thấp (từ GPIO của ESP32).

Vai trò trong hệ thống:

- Kênh 1: Điều khiển quạt của phòng 1 (bật/tắt).
- Kênh 2: Điều khiển quạt của phòng 2 (bật/tắt).

- Khi nồng độ khí vượt ngưỡng (chế độ AUTO) hoặc khi nhận lệnh từ người dùng (chế độ MANUAL), GPIO của ESP32 sẽ kích hoạt relay để cấp điện cho quạt.

Thông số kỹ thuật:

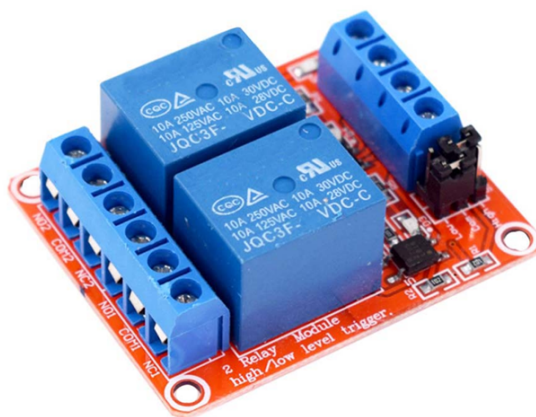
- Số kênh: 2 (hai relay độc lập).
- Cuộn dây: 5V DC.
- Tiếp điểm chính: Thường có khả năng chịu tải tới 10A / 250V AC hoặc 10A / 30V DC (tùy loại).
- Loại kích hoạt: Active LOW (cuộn dây được kích hoạt khi tín hiệu vào là mức thấp 0V).
- Cách ly: Sử dụng opto-coupler (opto-cách ly) để tách biệt mạch điều khiển (3.3V từ GPIO ESP32) khỏi mạch công suất (5V/240V).
- Dòng điều khiển: Khoảng 20-30 mA mỗi kênh.

Cấu hình trong dự án:

- Chân IN1 (Input kênh 1): GPIO 32 của ESP32 → Quạt phòng 1.
- Chân IN2 (Input kênh 2): GPIO 33 của ESP32 → Quạt phòng 2.
- Chân GND: Nối chung đất với ESP32 và nguồn 5V.
- Chân VCC: Cấp 5V từ hệ thống nguồn.
- Khởi tạo trong `setup()`: `pinMode(32, OUTPUT); pinMode(33, OUTPUT);` và đặt trạng thái ban đầu an toàn trước khi vào vòng điều khiển.

Hoạt động:

- ESP32 xuất tín hiệu điều khiển trên GPIO 32 và GPIO 33 để bật/tắt relay tương ứng từng phòng.
- Mức kích hoạt thực tế (HIGH hoặc LOW) phụ thuộc loại module relay, được chuẩn hóa trong hàm điều khiển của firmware.

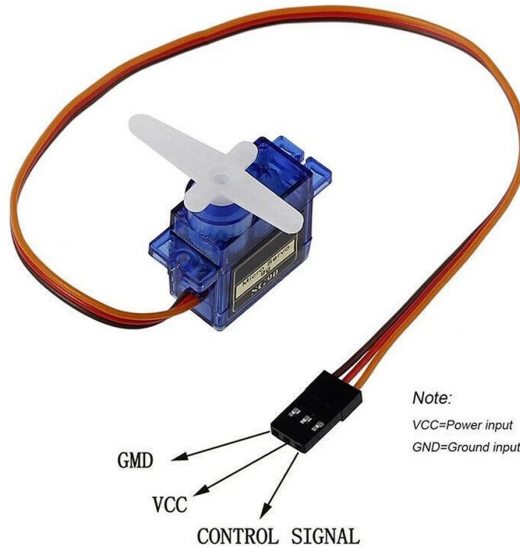


Hình 2.8: Module Relay 2 kênh 5V với sơ đồ chân kết nối

2.2.7 Servo SG90

a, Giới thiệu nhanh

Servo SG90 là loại động cơ servo siêu nhỏ (Micro Servo), cực kỳ phổ biến trong cộng đồng Maker và STEM. Nó nổi tiếng nhờ sự cân bằng giữa giá thành rẻ, kích thước nhỏ (khoảng 9 g) và khả năng điều khiển góc quay chính xác. Thường dùng với Arduino, ESP32 hoặc Raspberry Pi.



Hình 2.9: Servo SG90 (Micro Servo)

b, Chức năng & Ứng dụng

Khác với động cơ DC quay liên tục, Servo SG90 được thiết kế để duy trì một góc quay xác định (thường 0° đến 180°).

- **Chức năng:** Điều khiển vị trí góc của trục động cơ dựa trên tín hiệu xung PWM.
- **Ứng dụng:**
 - Khớp nối trong cánh tay robot (Robot Arm).
 - Điều khiển góc lái cho xe điều khiển từ xa (RC Car) hoặc máy bay mô hình.
 - Đóng/mở khóa cửa thông minh hoặc nắp thùng rác tự động.
 - Điều khiển góc quét cho cảm biến siêu âm tránh vật cản.

c, Thông số kỹ thuật chính

Trọng lượng	9 g
Kích thước	22.2 × 11.8 × 22.7 mm
Điện áp hoạt động	4.8 V – 6 V (ổn định nhất ở 5 V)
Lực kéo (Stall Torque)	1.2 kg/cm (tại 4.8 V) đến 1.6 kg/cm (tại 6 V)
Tốc độ phản ứng	0.1 s / 60°
Góc quay	0° – 180°
Chất liệu bánh răng	Nhựa (POM)

d, Sơ đồ chân (Pinout)

Servo SG90 có 3 dây màu chuẩn:

- Dây Nâu (Brown): GND.
- Dây Đỏ (Red): VCC (5 V).

- Dây Cam (Orange): Tín hiệu điều khiển (PWM Signal).

e, Nguyên lý hoạt động

Nguyên lý cốt lõi là Điều chế độ rộng xung (PWM). Bên trong servo có mạch điều khiển và biến trở gắn với trục đầu ra. Mạch so sánh vị trí hiện tại với độ rộng xung nhận được.

- Tần số hoạt động: thường 50 Hz (chu kỳ 20 ms).
- Độ rộng xung (High) quyết định góc:
 - 0.5 ms $\rightarrow$ 0°.
 - 1.5 ms $\rightarrow$ 90°.
 - 2.5 ms $\rightarrow$ 180°.

Lưu ý: Do bánh răng bằng nhựa, tránh ép trục khi không cấp điện hoặc tải vượt quá lực kéo.

2.2.8 Còi SFM-27

a, Giới thiệu nhanh

SFM-27 là còi báo động chủ động (Active Buzzer) sử dụng công nghệ thạch anh, có hai tai bắt vít hai bên để cố định lên vỏ. Hoạt động trong dải 3 V – 24 V, thích hợp cho mạch 5 V lẫn hệ thống 12 V/24 V.



Hình 2.10: Còi SFM-27 (Active Buzzer)

b, Chức năng & Ứng dụng

Mục đích chính là phát âm thanh cảnh báo cường độ cao.

- **Chức năng:** Phát tiếng "bíp" liên tục hoặc ngắt quãng khi có nguồn DC.
- **Ứng dụng:**
 - Hệ thống báo động chống trộm, báo cháy.
 - Cảnh báo lùi cho xe điện, robot.
 - Thông báo trạng thái trên bảng điều khiển máy công nghiệp.
 - Còi báo cho thiết bị đo lường.

c, Thông số kỹ thuật chính

Loại còi	Còi chủ động (Active Buzzer)
Điện áp hoạt động	3 V – 24 V DC
Dòng điện định mức	≤ 15 mA (tại 12 V)
Cường độ âm thanh	≥ 90 dB (ở 10 cm, tại 12 V)
Tần số cộng hưởng	3000 ± 500 Hz
Nhiệt độ hoạt động	-20°C đến +60°C
Kích thước	Đường kính thân 30 mm; khoảng cách 2 lỗ bắt vít 40 mm

d, Sơ đồ chân (Wiring)

- Dây Đỏ (+): Nối vào VCC (3V–24V).
- Dây Đen (-): Nối vào GND.

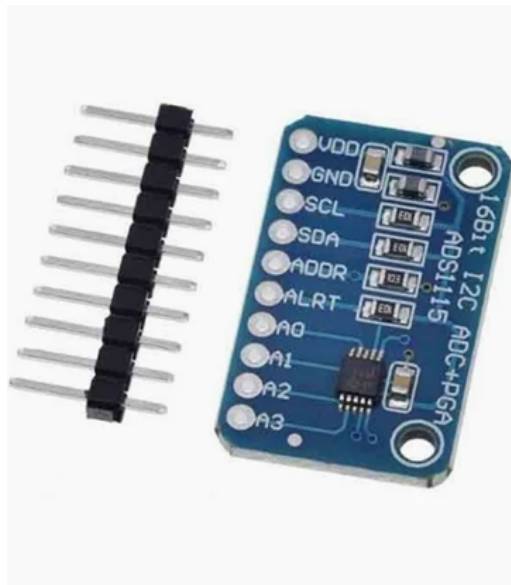
Ghi chú: Là còi chủ động nên chỉ cần cấp nguồn là hoạt động; để điều khiển bằng Arduino/ESP32 nên dùng transistor (ví dụ S8050) làm công tắc để tránh kéo quá tải chân IO.

e, Nguyên lý hoạt động

SFM-27 tích hợp mạch dao động và màng rung thạch anh (piezo) trong vỏ. Khi cấp điện, mạch tạo dao động xoay chiều cỡ 3000 Hz đưa vào màng thạch anh, tạo ra rung động và phát âm.

2.2.9 Module ADC ADS1115 16-bit 4 kênh**a, Giới thiệu nhanh**

ADS1115 là module ADC 16-bit, 4 kênh, giao tiếp I2C, tích hợp PGA cho phép khuếch đại tín hiệu yếu. Thích hợp cho đo lường chính xác mà ADC tích hợp vi điều khiển khó đạt được.



Hình 2.11: Module ADC ADS1115 16-bit 4 kênh (I2C)

b, Chức năng & Ứng dụng

- **Chức năng:** Chuyển đổi analog sang digital 16-bit; hỗ trợ đo đơn (single-ended) hoặc vi sai (differential).
- **Ứng dụng:** Đo loadcell, cảm biến nhiệt độ chính xác, giám sát điện áp pin, hệ thống Data Acquisition.

c, Thông số kỹ thuật chính

Độ phân giải	16 bits
Giao tiếp	I2C (hỗ trợ 4 địa chỉ khác nhau qua chân ADDR)
Điện áp hoạt động	2.0 V – 5.5 V
Dòng tiêu thụ	Rất thấp (chế độ liên tục 150 μ A)
Tốc độ lấy mẫu	8 SPS – 860 SPS (tùy cấu hình)
PGA	Lên đến x16
Số kênh	4 kênh đơn (A0–A3) hoặc 2 kênh vi sai

d, Sơ đồ chân (Pinout)

- VDD: Nguồn (2.0V–5.5V).
- GND: Mass.
- SCL / SDA: Đường I2C.
- ADDR: Chọn địa chỉ I2C (GND/VDD/SDA/SCL).
- ALRT: Ngắt/Comparator output (tùy dùng).
- A0–A3: 4 đầu vào analog.

e, Nguyên lý hoạt động

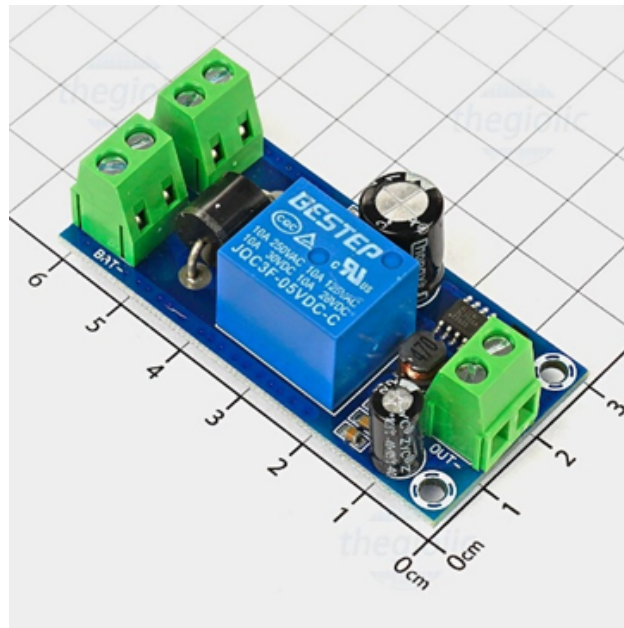
ADS1115 hoạt động dựa trên cấu trúc Delta-Sigma ($\Delta\Sigma$) ADC. Logic vận hành của nó diễn ra như sau:

1. **Thu nhận & Khuếch đại:** Tín hiệu Analog đi vào qua các chân A0–A3. Nếu tín hiệu quá yếu, bộ PGA (Programmable Gain Amplifier) bên trong sẽ khuếch đại nó lên (lên tới 16 lần) để đảm bảo độ chính xác.
2. **Chuyển đổi:** Bộ chuyển đổi Delta-Sigma lấy mẫu tín hiệu liên tục ở tần số cao, sử dụng các bộ lọc kỹ thuật số để triệt tiêu nhiễu tần số thấp và nhiễu nguồn (như nhiễu 50 Hz/60 Hz từ điện lưới).
3. **Số hóa:** Tín hiệu được chuyển thành giá trị số 16-bit. Với dải đo mặc định 6.144 V, mỗi đơn vị (LSB) tương đương với khoảng 187.5 μ V – một con số cực kỳ ấn tượng!
4. **Truyền tải:** Kết quả cuối cùng được lưu vào các thanh ghi nội bộ. Vi điều khiển (như ESP32 hoặc Arduino) sẽ gọi dữ liệu này thông qua giao thức I2C chỉ với 2 dây tín hiệu.

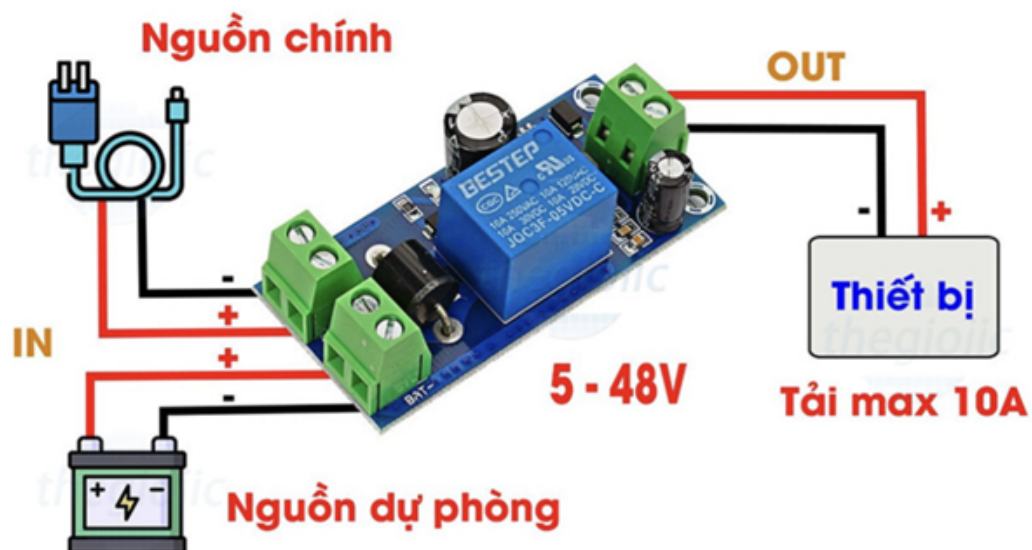
Mẹo nhỏ: Nếu bạn cần dùng nhiều module ADS1115 trên cùng một bo mạch, hãy tận dụng chân ADDR. Bằng cách nối chân này với các chân khác nhau, bạn có thể thiết lập tối đa 4 địa chỉ I2C khác nhau, cho phép đọc tới 16 kênh Analog chỉ trên một đường bus duy nhất!

2.2.10 Module YX850 mạch tự động chuyển nguồn dự phòng 5-48VDC

YX850 là một mạch tự động chuyển nguồn dự phòng được sử dụng để chuyển giữa hai nguồn chính và phụ (thường là nguồn pin, ắc quy) tự động để cấp cho hệ thống khi nguồn chính không còn hoạt động. Mạch sử dụng điện áp 5-48VDC, thích hợp cho các ứng dụng cần nguồn pin dự phòng.



Hình 2.12: Module YX850 thực tế



Hình 2.13: Nguyên lý chuyển nguồn chính và nguồn dự phòng của YX850

Mạch được thiết kế nhỏ gọn với cách sử dụng đơn giản nhưng vẫn đạt năng suất hiệu quả. Có thể sử dụng để cấp nguồn dự phòng cho hệ thống camera giám sát, hệ thống cửa ra vào kiểm soát, hệ thống báo động, hệ thống báo trộm.

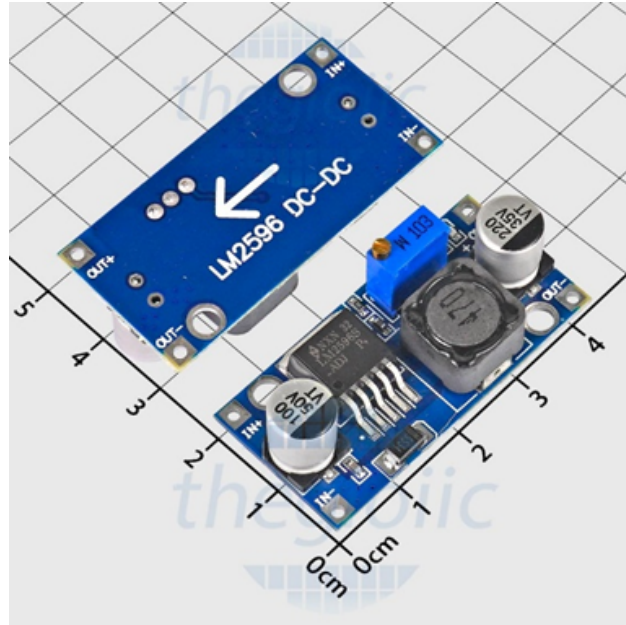
Thông số kỹ thuật:

- Điện áp: 5-48VDC.
- Khi có nguồn, nó được kết nối với pin để sạc.
- Tải: 10A.
- Kích thước sản phẩm: 61x30mm.

2.2.11 Module hạ áp LM2596

LM2596 là IC ổn áp hạ áp (Buck Converter) dòng cao 3A phổ biến, chuyển đổi điện áp DC cao xuống mức thấp hơn (1.23V-30V) với hiệu suất lên tới 92%. Nó có phiên bản cố định (3.3V, 5V, 12V) và điều

chỉnh được (ADJ), thường dùng trong các module nguồn nhỏ gọn cho dự án robot, camera và IoT.



Hình 2.14: Module hạ áp LM2596

Vì nguồn dự phòng là pin 3,7V đấu nối tiếp và vì mạch cần 5V, nên cần module hạ áp để điện áp ở ngưỡng 5V cho hệ thống sử dụng ổn định.

Thông số kỹ thuật chính của module LM2596:

- Điện áp đầu vào: 3V-30V/35V.
- Điện áp đầu ra: Điều chỉnh được từ 1.5V-30V/35V (phiên bản ADJ).
- Dòng điện đầu ra: 3A (liên tục), có thể cần tản nhiệt nếu dòng cao hơn 1A.
- Hiệu suất: Lên tới 92%.
- Tần số chuyển mạch: 150kHz (cho phép dùng linh kiện lọc nhỏ gọn). [1, 2, 3, 4, 5, 6]

Các đặc điểm nổi bật:

- Thiết kế đơn giản: Sử dụng rất ít linh kiện ngoại vi.
- Bảo vệ tích hợp: Có chức năng bảo vệ quá nhiệt và ngắn mạch.
- Ứng dụng: Mạch sạc pin, cấp nguồn cho vi điều khiển, camera, motor, loa mini.

2.2.12 Adapter 5V 2A

Chuyển đổi nguồn:



Hình 2.15: Adapter nguồn 5V 2A

- Biến đổi AC 220V (điện lưới) sang DC 5V.
- Đây là bộ nguồn AC-DC switching.

Cung cấp năng lượng ổn định:

- Cấp điện cho:
 - Vi điều khiển (ESP32, Arduino).
 - Sensor (MQ135).
 - Module (Relay, WiFi).

Thông số kỹ thuật quan trọng:

Thông số đầu vào (Input):

- Điện áp: 100-240V AC.
- Tần số: 50-60Hz.

Thông số đầu ra (Output):

- Điện áp: 5V DC.
- Dòng tối đa: 2A.

2.2.13 Nguồn dự phòng pin lithium 18650

Pin 18650 là loại pin sạc Lithium-ion (Li-ion) hình trụ, được sử dụng phổ biến trong các hệ thống nhúng và IoT.



Hình 2.16: Pin lithium-ion 18650 dùng làm nguồn dự phòng

- Ký hiệu 18650:
 - Đường kính: 18 mm.
 - Chiều dài: 65 mm.

Chức năng trong hệ thống:

Trong hệ thống, pin 18650 đóng vai trò:

- Nguồn dự phòng (backup power) khi mất điện từ adapter.
- Duy trì hoạt động cho:
 - ESP32.
 - Cảm biến MQ-135.
 - Relay và quạt.

Đảm bảo hệ thống giám sát không bị gián đoạn.

Thông số kỹ thuật chính:

- Điện áp: 3.7V.
- Dung lượng: xấp xỉ 1500 mAh (tùy loại thực tế).

2.2.14 Bảng cấu hình chân GPIO

Bảng dưới đây tổng hợp toàn bộ các chân GPIO và các linh kiện được kết nối trong hệ thống:

Bảng 2.1: Cấu hình chân GPIO của ESP32 trong hệ thống

Chức năng	Chân	Loại	Linh kiện
Khí phòng 1	A0	I2C ADC	MQ-135 #1
Khí phòng 1	GND	GND	GND
Khí phòng 2	A1	I2C ADC	MQ-135 #2
Khí phòng 2	GND	GND	GND
Relay 1	GPIO 32	Output	Relay IN1
Relay 2	GPIO 33	Output	Relay IN2
I2C SDA	GPIO 21	I2C SDA	LCD, ADS1115
I2C SCL	GPIO 22	I2C SCL	LCD, ADS1115
Servo phòng 1	GPIO 26	PWM	SG90 #1
Servo phòng 2	GPIO 27	PWM	SG90 #2
Buzzer phòng 1	GPIO 25	Output	SFM-27 #1
Buzzer phòng 2	GPIO 4	Output	SFM-27 #2
Nút khẩn cấp 1	GPIO 34	Input ISR	EMG R1
Nút khẩn cấp 2	GPIO 35	Input ISR	EMG R2
Nút khẩn cấp chung	GPIO 23	Input ISR	EMG ALL
Nguồn chính	5V/GND	Power	Adapter 5V 2A
Nguồn 3.3V	3V3	Power	CP2102/CH340

2.3 Phần mềm của hệ thống

2.3.1 Tổng quan chức năng phần mềm

Hệ thống phần mềm trên ESP32 thực hiện các chức năng:

- **Khởi động hệ thống:** Đồng bộ thời gian từ server NTP (`pool.ntp.org`, `time.nist.gov`, múi giờ GMT+7) để đảm bảo timestamp dữ liệu telemetry chính xác cho việc lưu trữ lịch sử và vẽ biểu đồ.
- **Đọc dữ liệu từ 2 cảm biến MQ-135 qua ADS1115:** Đọc ADC 16-bit từ ADS1115 (kênh A0, A1), sau đó ánh xạ về thang 12-bit (0-4095) để giữ tương thích với các ngưỡng AQI đã hiệu chỉnh.
- **Lọc nhiễu bằng EWMA:** Áp dụng bộ lọc Exponential Weighted Moving Average (hệ số $\alpha = 0.2$) để làm mượt dao động ngẫu nhiên của ADC do nhiễu nguồn điện và sự biến đổi nhanh của cảm biến, tránh kích hoạt relay bất thành linh khi giá trị dao động quanh ngưỡng.
- **Phát hiện lỗi cảm biến:** Kiểm tra xem giá trị lọc có nằm trong dải hợp lệ $[MQ_MIN, MQ_MAX] = [50, 3800]$. Nếu vượt ra ngoài dải, đánh dấu trường `error = true` và gán `"sensor": "ERR"` vào payload MQTT để thông báo cho backend biết thiết bị cảm biến gặp vấn đề.
- **Phân loại chất lượng không khí:** Dựa trên giá trị lọc, phân loại mức độ ô nhiễm thành 4 cấp: GOOD, MOD (Moderate), BAD, DANG (Dangerous).
- **Gửi dữ liệu lên HiveMQ Broker:** Định kỳ (mỗi 3 giây) gửi gói tin dữ liệu định dạng JSON chứa các thông tin: nồng độ khí, mức ô nhiễm, trạng thái quạt, chế độ hoạt động, trạng thái lỗi thông qua giao thức MQTT qua kết nối TLS an toàn (port 8883).
- **Nhận lệnh điều khiển từ server:** Subscribe vào topic MQTT để lắng nghe các lệnh từ người dùng hoặc hệ thống điều khiển từ xa, bao gồm:
 - Lệnh chuyển chế độ: AUTO hoặc MANUAL.
 - Lệnh điều khiển quạt, còi và góc mở cửa sổ: `fan`, `buzzer`, `window`.
- **Điều khiển quạt tương ứng từng phòng:**
 - Chế độ AUTO: Hệ thống tự động kích hoạt relay để bật quạt khi nồng độ vượt ngưỡng, tắt khi nồng độ hạ xuống.

– Chế độ MANUAL: Lệnh từ người dùng được ưu tiên cao nhất, ghi đè lên logic tự động.

- **Hiển thị trên LCD I2C 16×2:** Xoay vòng hiển thị thông tin các phòng mỗi 2 giây (paging), bao gồm: nồng độ khí, mức cảnh báo, trạng thái quạt, chế độ điều khiển.
- **Cơ chế chống nhiễu I2C khi Relay đóng/ngắt:** Khi có sự kiện chuyển trạng thái quạt (relay đóng/ngắt), dòng cảm ứng ngược từ cuộn relay có thể gây nhiễu lan ra bus I2C, khiến LCD bị treo. Để xử lý, task LCD sẽ re-initialize I2C bus (`Wire.begin()`) và LCD khi phát hiện cờ `flagResetI2C`, bảo đảm giao tiếp I2C được khôi phục.
- **Xử lý khẩn cấp tại hiện trường:** Hệ thống có 3 nút bấm vật lý để bật/tắt trạng thái EMERGENCY theo từng phòng hoặc toàn hệ thống, được xử lý bằng ngắt ngoài và task ưu tiên cao nhất.

2.3.2 Phân ngưỡng chất lượng không khí (AQI Classification)

Hệ thống phân loại mức độ ô nhiễm dựa trên giá trị ADC 12-bit (0–4095) đã lọc từ cảm biến MQ-135. Các ngưỡng được chọn dựa trên khảo sát thực nghiệm trong môi trường làm việc:

Bảng 2.2: Bảng phân ngưỡng chất lượng không khí (AQI) và hành động điều khiển

Giá trị raw ADC	Mức	Ý nghĩa	Hành động AUTO
< 1200	GOOD	Tốt	Tắt quạt
1200 – 1999	MOD	Trung bình	Tắt quạt
2000 – 2999	BAD	Kém	Bật quạt
≥ 3000	DANG	Nguy hiểm	Bật quạt

Lưu ý quan trọng: Giá trị này là raw ADC 12-bit chứ chưa được quy đổi sang ppm (parts per million). Việc quy đổi MQ-135 sang ppm yêu cầu Calibration đường cong R_0/R_s phức tạp và vượt quá phạm vi của bài tập lớn này. Nhóm sử dụng raw ADC làm chỉ số tương đối để so sánh ngưỡng và xác định tình trạng ô nhiễm tại vị trí đặt cảm biến.

2.3.3 Bộ lọc EWMA (Exponential Weighted Moving Average) và Phát hiện lỗi cảm biến

a, Bộ lọc EWMA

Trong thực tế, giá trị ADC của cảm biến thường dao động do nhiễu từ:

- Nguồn điện 5V dùng chung với motor/relay (dòng cảm ứng, sụt áp tức thời).
- Sự nhạy cảm tự nhiên của cảm biến bán dẫn với biến thiên nhiệt độ, độ ẩm.
- Bản chất của ADC khi lấy mẫu với SNR (Signal-to-Noise Ratio) hạn chế.

Để xử lý, hệ thống áp dụng bộ lọc Exponential Weighted Moving Average (EWMA) với công thức:

$$\text{filtered_value}_n = \alpha \cdot \text{raw_value}_n + (1 - \alpha) \cdot \text{filtered_value}_{n-1}$$

Trong đó:

- $\alpha = 0.2$ (hệ số làm mượt, chọn thủ công qua thử nghiệm).
- raw_value_n : giá trị ADC vừa đọc.
- $\text{filtered_value}_{n-1}$: giá trị lọc từ lần đọc trước.

Mục đích:

- Làm mượt các dao động ngẫu nhiên, làm cho quyết định kích hoạt relay ổn định hơn.
- Tránh hiện tượng quạt bị "trip" (bật/tắt liên tục) khi giá trị dao động quanh ngưỡng.
- Giữ tính responsive của hệ thống (vẫn phản hồi nhanh với những thay đổi lớn trong nồng độ khí).

b, Phát hiện lỗi cảm biến

Cảm biến MQ-135 có thể gặp lỗi do:

- Tầm nhiệt bị hỏng hoặc không sấy đủ (khi vừa khởi động).
- Chân A0 bị ngắn/mở.
- Độ ẩm hoặc hóa chất khắc nghiệt tấn công linh kiện.

Cơ chế phát hiện:

- Sau khi lọc EWMA, giá trị được kiểm tra xem có nằm trong dải hợp lệ: $MQ_MIN = 50$, $MQ_MAX = 3800$.
- Nếu giá trị < 50 hoặc > 3800 , cảm biến được đánh dấu có lỗi (`error = true`).
- Khi có lỗi, payload MQTT sẽ gán `"sensor": "ERR"` thay vì giá trị ADC, cho phép backend phát hiện và ghi log sự cố này.
- Quạt sẽ KHÔNG tự động bật khi cảm biến lỗi (chế độ AUTO) để tránh hành động bất thường; người dùng có thể điều khiển thủ công nếu cần.

2.3.4 Cơ chế chống nhiễu I2C khi Relay đóng/ngắt

Vấn đề thực tế gặp phải:

- Khi relay đóng/ngắt cuộn cảm của quạt, dòng điện chột thay đổi (đặc biệt với quạt chạy bằng DC motor).
- Hiệu ứng cảm ứng ngược (back-EMF) tạo ra xung áp cao, lan truyền qua các dây dẫn xung quanh bus I2C.
- Bus I2C là bus kéo lên (pull-up) với tần số cao (100 kHz ở Standard mode, 400 kHz ở Fast mode), rất nhạy cảm với nhiễu.
- Kết quả: LCD bị "treo" (không phản hồi), hiển thị ký tự rác hoặc mất kết nối I2C.

Giải pháp triển khai:

- Trong hàm `executeFanControl()`, ngay sau khi gọi `digitalWrite()` để thay đổi trạng thái relay, đặt cờ `flagResetI2C = true`.
- Task LCD (chạy với chu kỳ 2 giây) kiểm tra cờ này. Nếu điều kiện là true, task sẽ thực hiện:
 1. Gọi `Wire.end()` để ngắt kết nối I2C.
 2. Gọi `Wire.begin(SDA_PIN, SCL_PIN)` để khởi tạo lại I2C bus.
 3. Gọi `lcd.init()` và `lcd.backlight()` để re-initialize LCD.
 4. Đặt `flagResetI2C = false`.
- Quá trình này mất khoảng 10–50 ms, không ảnh hưởng đáng kể đến tốc độ refresh trên LCD.

Đây là một điểm quan trọng chứng minh nhóm đã thử nghiệm thực tế và xử lý các lỗi kỹ thuật gặp phải trong quá trình lắp ráp hệ thống.

2.3.5 Đồng bộ thời gian NTP

Hệ thống cần timestamp chính xác cho các mục đích:

- Gắn nhãn thời gian vào mỗi dữ liệu telemetry gửi lên server (để tạo biểu đồ lịch sử chính xác).
- Ghi log các sự kiện (kích hoạt alarm, thay đổi chế độ, lỗi).
- Đồng bộ với timezone của khu vực hoạt động (GMT+7 cho Việt Nam).

Cách triển khai:

- Trong hàm `setup()`, ESP32 gọi:
 - `configTime(7*3600, 0, "pool.ntp.org", "time.nist.gov");`
 - Tham số đầu tiên là offset múi giờ (7 giờ = GMT+7).
 - Tham số 0 là offset daylight saving (không áp dụng cho Việt Nam).
 - Hai tham số cuối là NTP server backup.
- ESP32 sẽ đợi tối đa 10 giây để nhận phản hồi từ NTP server.
- Nếu thành công, giờ hệ thống sẽ được đặt. Nếu thất bại, ESP32 vẫn hoạt động nhưng timestamp sẽ không chính xác.
- Để lấy giờ hiện tại trong chương trình, sử dụng:
 - `time_t now = time(nullptr);` hoặc
 - `struct tm timeinfo; localtime_r(&now, &timeinfo);`

2.3.6 Giao thức truyền tin MQTT

a, Cấu hình Broker MQTT

- **Server:** HiveMQ Cloud (21b69e31ed5e...hivemq.cloud).
- **Port:** 8883 (MQTT over TLS/SSL, kết nối an toàn).
- **Xác thực:** Username và Password cấu hình trong firmware. Lưu ý: mật khẩu hiển thị trong mã - cần che dấu thực tế.
- **Client ID:** ESP32_<MAC_ADDRESS> để đảm bảo duy nhất.
- **Keep-Alive:** 60 giây.
- **Buffer size:** 1024 byte cho publish/subscribe payload.

b, Bảo mật TLS

- Sử dụng `WiFiClientSecure` thay vì `WiFiClient`.
- Gọi `client.setInsecure()`; để bỏ qua xác thực CA (bộ nhớ ESP32 giới hạn).
- Phương pháp này giảm bớt bảo mật nhưng vẫn mã hóa dữ liệu trên đường truyền.
- Sản phẩm thực tế nên tích hợp certificate CA để xác thực server.

c, Cấu trúc Topic MQTT

Hệ thống sử dụng hai bộ topic: chuẩn mới (với device ID) và legacy (để tương thích):

Bảng 2.3: Cấu trúc Topic MQTT

Topic	Hướng	QoS	Mô tả
air/data/{id}	ESP32 → Backend	0	JSON telemetry
air/data	ESP32 → Backend	0	Legacy telemetry
air/control/{id}	Backend → ESP32	1	JSON command
air/control	Backend → ESP32	1	Legacy command
air/updatefirmware	Backend → ESP32	1	OTA (URL+ver)
air/firmwareupdatestatus	ESP32 → Backend	0	OTA status

Payload Telemetry (ESP32 publish lên, mỗi 3 giây, QoS 0):

```
{
```

```

"rooms": [
  {
    "id": 1,
    "value": 1450,
    "level": "MOD",
    "emergency": false,
    "mode": "AUTO",
    "fan": false,
    "buzzer": false,
    "window": 45,
    "sensor": "OK"
  },
  {
    "id": 2,
    "value": 2150,
    "level": "BAD",
    "emergency": false,
    "mode": "MANUAL",
    "fan": true,
    "buzzer": false,
    "window": 90,
    "sensor": "OK"
  }
]
}

```

Payload Control (Backend gửi xuống, QoS 1):

```

{
  "room": 1,
  "mode": "MANUAL",
  "fan": false,
  "buzzer": false,
  "window": 90
}

```

Lưu ý: Payload control gửi đủ trường bắt buộc `room`, `mode`; các trường `fan`, `buzzer`, `window` là tùy chọn và chỉ có hiệu lực khi `mode = MANUAL`.

2.3.7 Thiết kế không sử dụng RTOS (mô hình đối chiếu)

Phần này trình bày kiến trúc lý thuyết để đối chiếu. Phiên bản triển khai thực tế của hệ thống sử dụng FreeRTOS, được mô tả trong mục kế tiếp.

a, Phân chia Task (Logical Tasks)

Hệ thống được chia thành các tác vụ logic:

Bảng 2.4: Phân chia tác vụ logic khi không sử dụng RTOS

Task	Chức năng
Sensor Task	Đọc dữ liệu 2 MQ-135
Processing Task	Xử lý, phân loại mức khí
MQTT Task	Gửi dữ liệu lên broker
Control Task	Điều khiển quạt
Command Task	Nhận lệnh từ server

b, Scheduling (Lập lịch)

Sử dụng cooperative scheduling (vòng lặp loop) với millis():

```
void loop () {
    readSensorTask ();
    processTask ();
    mqttPublishTask ();
    mqttReceiveTask ();
    controlTask ();
}
```

Cách triển khai:

- Mỗi task có timer riêng.
- Không dùng delay().

Ví dụ:

```
if ( millis () - lastRead >= 1000) {
    readSensor ();
}
```

c, FSM - Máy trạng thái hữu hạn

Mỗi phòng có FSM riêng.

Trạng thái chính:

- MODE:
 - AUTO.
 - MANUAL.
- FAN STATE:
 - OFF.
 - ON.

d, Logic điều khiển

Nguyên tắc xử lý Auto và Manual:

Bảng 2.5: Nguyên tắc điều khiển Auto và Manual

Mode	Điều khiển
AUTO	Hệ thống tự quyết theo ngưỡng
MANUAL	Người dùng điều khiển hoàn toàn

e, Luồng xử lý lệnh từ server

Bước 1: Nhận lệnh setup.

Ví dụ MQTT:

```
{
  "room": 1,
  "mode": "MANUAL"
}
```

ESP32:

- Cập nhật mode.
- Reset trạng thái điều khiển.

Bước 2: Nếu là MANUAL thì nhận lệnh điều khiển.

```
{
  "room": 1,
  "fan": true
}
```

f, Pseudocode FSM

```
if (room.mode == AUTO) {
    if (airQuality > threshold) {
        fan = ON;
    } else {
        fan = OFF;
    }
}

if (room.mode == MANUAL) {
    fan = userCommand;
}
```

g, Xử lý ngoại lệ

Vấn đề:

- Xung đột giữa Auto và Manual.

Giải pháp:

- Khi nhận lệnh chuyển mode:
 - Ưu tiên tuyệt đối MANUAL.
 - Auto bị disable hoàn toàn.

```
if (modeChangedToManual) {
    disableAutoControl();
}
```

2.3.8 Thiết kế có sử dụng RTOS (FreeRTOS trên ESP32)

a, Phân chia Task (Real Tasks)

Hệ thống sử dụng 6 task chạy song song trên FreeRTOS. Bảng dưới trình bày chi tiết từng task:

Bảng 2.6: Phân chia tác vụ khi sử dụng RTOS trên ESP32

Task	Stack	Pri	Chu kỳ	Chức năng chính
taskEmergency	3 KB	6	Sự kiện	ISR nút khẩn cấp, Emergency
taskOTA	8 KB	5	Động	OTA, ghi flash
taskControl	3 KB	4	1.5 s	FSM: Emergency > DANG > MANUAL > AUTO
taskSensor	2 KB	3	1 s	ADC, EWMA, phát hiện lỗi, AQI
taskLCD	2 KB	2	2 s	LCD paging, reset I2C
taskMQTT	8 KB	1	50 ms/3 s	WiFi/MQTT, control/telemetry

Giải thích chi tiết:

- **Stack Size:** taskMQTT và taskOTA dùng 8 KB do cần buffer lớn cho TLS/OTA. Các task còn lại tối ưu 2-3 KB.
- **Priority:** taskEmergency có ưu tiên cao nhất để đáp ứng an toàn tức thời. taskOTA cao hơn taskControl để tránh gián đoạn khi đang cập nhật firmware.
- **Period:** Các task vận hành theo chu kỳ riêng; riêng taskEmergency là event-driven (không polling), còn taskOTA chỉ tồn tại trong lúc cập nhật.
- **Chức năng:** Mô tả nhiệm vụ của mỗi task (nêu chi tiết ở các subsection sau).

b, Kiến trúc Task

Sensor Task/Emergency Task/MQTT Callback → Notify → Control Task

↓

LCD Task (I2C) và MQTT Task (truyền thông)

c, Quản lý tài nguyên**Mutex**

- dataMutex: Bảo vệ dữ liệu trạng thái phòng (mode, fan, buzzer, window, level, emergency).
- i2cMutex: Bảo vệ bus I2C dùng chung LCD và ADS1115.
- mqttMutex: Bảo vệ thao tác publish MQTT khi nhiều task cùng cần gửi.

```
xSemaphoreTake (dataMutex , portMAX_DELAY);
// cap nhap rooms[]
xSemaphoreGive (dataMutex);
```

Task Notification

- ISR và callback MQTT đánh thức trực tiếp taskControl bằng xTaskNotifyGive() thay cho polling.

d, FSM trong RTOS

FSM vẫn giữ nguyên logic nhưng chạy trong Control Task.

e, Xử lý MQTT

Khi nhận lệnh:

```
void mqttCallback (...) {
    parseCommand();
    xTaskNotifyGive (controlTaskHandle);
}
```

f, Xử lý Auto và Manual (RTOS version)

Nguyên tắc:

- Control Task xử lý theo thứ tự ưu tiên: EMERGENCY > DANG-Lockout > MANUAL > AUTO.
- Khi phòng ở trạng thái EMERGENCY, hệ thống bỏ qua điều khiển từ xa để bảo đảm an toàn hiện trường.

```
if (room.isEmergency) {
    forceEmergencyActuators();
} else if (room.mode == MANUAL) {
    manualControl();
} else {
    autoControl();
}
```

g, Xử lý ngoại lệ (chuẩn RTOS)

Vấn đề:

- Race condition khi:
 - MQTT nhận lệnh.
 - Control Task và Emergency Task đang xử lý đồng thời.

Giải pháp:

- Dùng 3 mutex chuyên biệt và task notification.
- Không giữ đồng thời hai mutex để tránh deadlock.

h, Ưu điểm RTOS

- Task chạy song song.
- Không block.
- Dễ mở rộng nhiều phòng.

2.3.9 Cơ chế điều khiển từ người dùng

Flow chuẩn:

1. User chọn phòng trên Dashboard.
2. User chọn chế độ:
 - AUTO: Hệ thống tự động kích hoạt quạt theo ngưỡng nồng độ khí.
 - MANUAL: User điều khiển quạt thủ công.
3. Backend gửi lệnh MQTT (payload đầy đủ):

```
{
  "room": 1,
  "mode": "MANUAL",
  "fan": false,
  "buzzer": false,
  "window": 90
}
```

4. ESP32 nhận lệnh, cập nhật trạng thái của phòng 1.

5. Payload gồm trường bắt buộc (`room, mode`) và trường tùy chọn (`fan, buzzer, window`) khi ở chế độ MANUAL.
6. Nếu phòng đang ở trạng thái EMERGENCY, lệnh điều khiển từ xa bị từ chối để bảo đảm an toàn.

2.3.10 Hệ thống khẩn cấp (Emergency Mode)

a, Mục tiêu và cơ chế kích hoạt

Hệ thống bổ sung cơ chế xử lý khẩn cấp bằng 3 nút bấm vật lý để người vận hành can thiệp ngay tại hiện trường:

- `BTN_EMG_R1` (GPIO 34): Toggle trạng thái khẩn cấp của phòng 1.
- `BTN_EMG_R2` (GPIO 35): Toggle trạng thái khẩn cấp của phòng 2.
- `BTN_EMG_ALL` (GPIO 23): Toggle đồng thời cả hai phòng.

b, ISR và debounce

Ba nút bấm được gắn ngắt cạnh xuống (FALLING). Trong ISR, firmware dùng debounce 250 ms, đặt cờ sự kiện và gọi `vTaskNotifyGiveFromISR()` để đánh thức `taskEmergency` ngay lập tức.

c, Logic ưu tiên trong điều khiển

Trong `taskControl`, thứ tự ưu tiên xử lý là:

1. **EMERGENCY:** ép còi ON, quạt ON, cửa sổ mở 180 độ.
2. **DANG-Lockout:** khi mức DANG ở AUTO, hệ thống ép về MANUAL an toàn cho tới khi người vận hành xác nhận lại.
3. **MANUAL:** chấp hành lệnh người dùng từ Dashboard.
4. **AUTO:** điều khiển theo ngưỡng AQI.

2.3.11 Watchdog Timer và khôi phục trạng thái

a, Task Watchdog

Hệ thống bật Task WDT timeout 30 giây để giám sát tính sống của các task chính. Mỗi task gọi `esp_task_wdt_reset()` theo chu kỳ để xác nhận vẫn hoạt động.

b, Khôi phục trạng thái sau reset

Các trạng thái quan trọng (`mode, fan, buzzer, góc servo, emergency`) được lưu trên RTC memory. Nếu reset do WDT/panic, hệ thống khôi phục lại trạng thái trước đó để tránh mất trạng thái an toàn.

2.3.12 Cơ chế cập nhật firmware OTA

a, Luồng OTA

Quy trình OTA: nhận lệnh, tải firmware, ghi flash, publish trạng thái, khởi động lại.

b, Ràng buộc an toàn OTA

Firmware từ chối OTA khi có phòng EMERGENCY; các task khác giảm tải khi ghi flash.

2.4 Phương pháp thực hiện và kết quả

2.4.1 Phương pháp thực hiện

Quá trình hiện thực hóa hệ thống được thực hiện qua các bước cụ thể sau:

a, Triển khai phần cứng

- **Lắp ráp mạch nguồn:** Adapter 5V 2A làm nguồn chính, YX850 quản lý nguồn dự phòng (pin 18650), LM2596 ổn định 5V.
- **Kết nối cảm biến và cơ cấu chấp hành:** 2 MQ-135 vào ADS1115 (A0, A1); Relay qua GPIO 32/33;

Servo SG90, Buzzer SFM-27 cập nhật trạng thái.

- **Thi công mô hình:** Để đảm bảo tính linh hoạt trong quá trình phát triển và kiểm tra hệ thống giám sát không khí nhà máy, nhóm đã tiến hành lắp ráp mô hình theo dạng thực nghiệm (prototype) trên nền tảng breadboard và các module rời. Các bước thi công cụ thể như sau:

i. **Bố trí linh kiện tổng thể:**

- **Khối điều khiển trung tâm:** Vi điều khiển ESP32 và 02 cảm biến MQ-135 được cắm trực tiếp trên Breadboard (testboard) lớn. Cách bố trí này giúp thu gọn diện tích và dễ dàng thay đổi sơ đồ nối dây khi cần thiết.
- **Khối chấp hành:** Module Relay 2 kênh được đặt tách biệt, kết nối qua dây jumper để điều khiển 02 quạt tản nhiệt 5V (mô phỏng hệ thống thông gió tại 2 khu vực khác nhau trong nhà máy).
- **Khối quản lý năng lượng:** Các module nguồn được sắp xếp theo trình tự luồng điện để tối ưu hóa không gian, bao gồm: Jack DC input, mạch chuyển nguồn dự phòng YX850, mạch hạ áp LM2596 và khay pin Lithium 18650.

ii. **Quy trình kết nối và đi dây (Wiring):**

- **Kết nối cảm biến:** 02 cảm biến MQ-135 được cấp nguồn 5V và nối tín hiệu analog vào ADS1115; ADS1115 giao tiếp với ESP32 qua bus I2C (SDA=21, SCL=22).
- **Kết nối điều khiển quạt:** Các chân GPIO điều khiển từ ESP32 được dẫn tới đầu vào (Input) của module Relay. Quạt được nối vào tiếp điểm của Relay, sử dụng nguồn 5V trực tiếp từ hệ thống nguồn chung.
- **Thiết lập hệ thống nguồn nối tầng:**
 1. Nguồn từ Adapter (qua Jack DC) và nguồn từ Pin 18650 cùng đi vào mạch YX850 để thực hiện chức năng tự động chuyển đổi khi mất điện.
 2. Đầu ra của YX850 được nối tới mạch hạ áp LM2596 để điều chỉnh điện áp về mức 5V ổn định.
 3. Nguồn 5V sau hạ áp được dẫn tới breadboard để nuôi ESP32, các cảm biến và cấp điện cho Relay/Quạt.

iii. **Hoàn thiện thực nghiệm:**

- Toàn bộ hệ thống dây dẫn được sử dụng là dây Jumper (Đực-Cái và Đực-Đực), các mối nối được kiểm tra kỹ lưỡng để đảm bảo độ tiếp xúc tốt và không gây sụt áp cho cảm biến.
- Các module được cố định tạm thời trên mặt phẳng thực nghiệm, đảm bảo khoảng cách an toàn giữa khối nguồn và khối điều khiển để tránh nhiễu điện từ.

b, Phát triển phần mềm

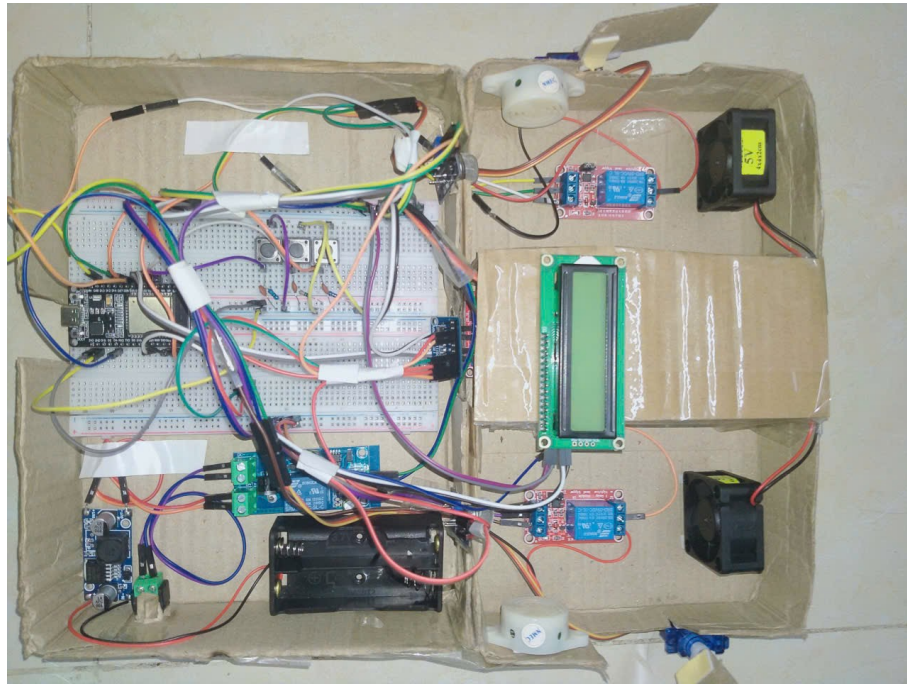
- **Cấu hình môi trường:** Lập trình trên nền tảng Arduino IDE cho ESP32, sử dụng các thư viện chuyên dụng:
 - `WiFi.h` và `WiFiClientSecure.h` cho kết nối WiFi an toàn.
 - `PubSubClient.h` cho giao thức MQTT.
 - `LiquidCrystal_I2C.h` cho điều khiển LCD.
 - `time.h` và `sys/time.h` cho đồng bộ NTP.
 - `freertos/FreeRTOS.h` để sử dụng RTOS.

- **Khởi tạo hệ thống:** Trong `setup()` :
 - Cấu hình các chân GPIO cho relay, servo, buzzer và các nút bấm khẩn cấp.
 - Khởi tạo I2C bus (`SDA=21`, `SCL=22`), LCD 16x2 và ADS1115 dùng chung bus.
 - Kết nối WiFi tới access point.
 - Đồng bộ thời gian từ NTP (`pool.ntp.org`, `time.nist.gov`, GMT+7).
 - Khởi tạo MQTT client với kết nối TLS an toàn (port 8883).
- **Xây dựng logic điều khiển:**
 - Hiện thực hóa máy trạng thái hữu hạn (FSM) để quản lý các chế độ AUTO (tự động theo ngưỡng) và MANUAL (người dùng điều khiển).
 - Thiết lập kết nối MQTT over TLS (WiFiClientSecure) để truyền nhận dữ liệu với HiveMQ Broker.
 - Áp dụng bộ lọc EWMA ($\alpha = 0.2$) để làm mượt dữ liệu ADC từ cảm biến.
 - Phát hiện lỗi cảm biến bằng cách kiểm tra dải hợp lệ [50, 3800].
 - Phân loại mức độ ô nhiễm thành 4 cấp (GOOD, MOD, BAD, DANG).
- **Phân chia Task (nếu dùng RTOS):**
 - 6 task: `taskEmergency`, `taskOTA` (tạo động), `taskControl`, `taskSensor`, `taskLCD`, `taskMQTT`.
 - Sử dụng 3 mutex chuyên biệt: `dataMutex`, `i2cMutex`, `mqttMutex`.
 - Sử dụng flag volatile `flagResetI2C` để đồng bộ việc khởi tạo lại bus I2C sau khi relay đóng/ngắt.
- **Callback MQTT:** Xử lý lệnh từ server khi ESP32 nhận được thông điệp trên topic `air/control` hoặc `air/updatefirmware`. Payload điều khiển được parse ra các trường (`room`, `mode`, `fan`, `buzzer`, `window`).

2.4.2 Kết quả đạt được

a, Sản phẩm thực tế

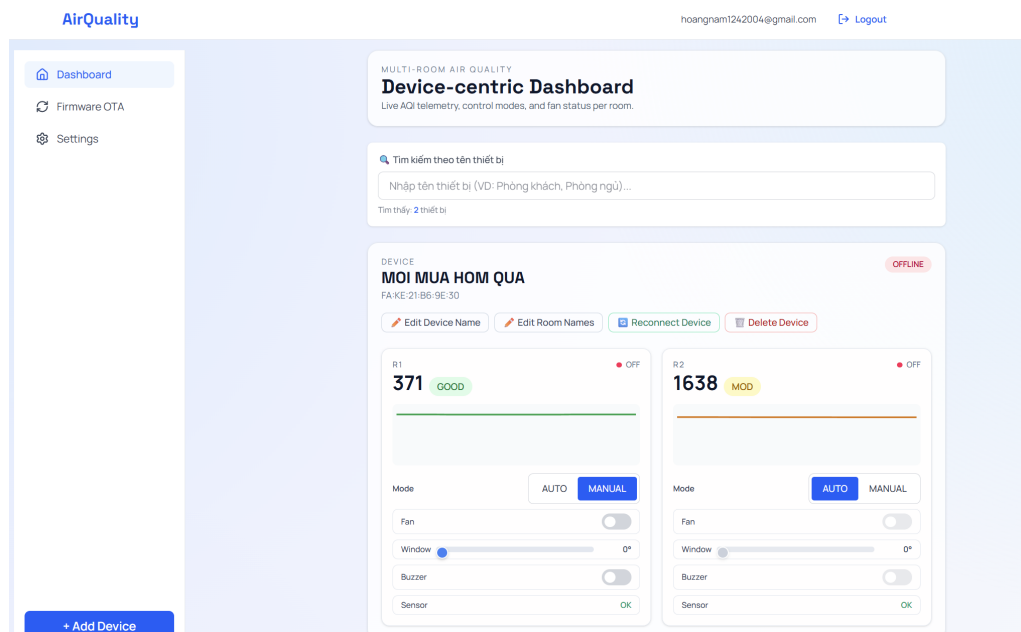
- Hệ thống đã được lắp ráp hoàn chỉnh thành một thiết bị giám sát IoT nhỏ gọn.



Hình 2.17: Hệ thống giám sát chất lượng không khí trong nhà máy

b, Giao diện giám sát và điều khiển

- Dữ liệu raw ADC và mức cảnh báo từ hai phòng được cập nhật liên tục lên Dashboard của Web/Mobile App dưới dạng biểu đồ và chỉ số trực quan.
- Người dùng có thể theo dõi trạng thái quạt, còi và chế độ hoạt động trực tiếp trên giao diện.



Hình 2.18: Giao diện website giám sát và điều khiển hệ thống

c, Thử nghiệm khả năng vận hành

- **Chế độ Auto:** Khi mô phỏng nồng độ khí cao (sử dụng bật lửa hoặc dung dịch cồn), cảm biến MQ-135 phản hồi nhanh chóng, ESP32 tự động kích hoạt quạt và còi khi nồng độ vượt mức ngưỡng quy định.

- **Chế độ Manual:** Lệnh điều khiển từ App được thực thi với độ trễ thấp (thông thường < 1 giây), đồng thời có quyền ưu tiên cao nhất, ghi đè lên logic tự động.
- **Nguồn dự phòng:** Khi ngắt nguồn Adapter, mạch YX850 tự động chuyển sang dùng pin 18650, đảm bảo hệ thống giám sát không bị gián đoạn.

2.4.3 Đánh giá và nhận xét

Dựa trên các tiêu chí tại, nhóm đưa ra các đánh giá sau:

- **Độ chính xác:** Cảm biến MQ-135 đáp ứng tốt việc phát hiện sự thay đổi chất lượng không khí trong bối cảnh nhà máy.
- **Tính thời gian thực:** Việc áp dụng cơ chế lập lịch (Scheduling) hoặc RTOS giúp hệ thống xử lý đồng thời việc đọc dữ liệu, cập nhật LCD và phản hồi lệnh từ xa một cách mượt mà.
- **Tính ổn định:** Hệ thống hoạt động ổn định trong thời gian dài thử nghiệm, kết nối MQTT tin cậy nhờ khả năng xử lý của ESP32 và mạch nguồn dự phòng.

CHƯƠNG 3. KẾT LUẬN

3.1 Kết luận

Sau quá trình nghiên cứu và triển khai, nhóm đã xây dựng thành công mô hình hệ thống giám sát chất lượng không khí đáp ứng đầy đủ các mục tiêu đề ra. Hệ thống không chỉ dừng lại ở mức mô hình thử nghiệm mà đã thể hiện được tư duy thiết kế của một sản phẩm IoT hoàn chỉnh.

Các kết quả nổi bật:

- Dữ liệu raw ADC và trạng thái hệ thống được cập nhật liên tục với độ trễ thấp qua giao thức MQTT.
- Cơ chế điều khiển tự động phản hồi chính xác khi môi trường thay đổi, đồng thời hỗ trợ quạt, còi và chế độ khẩn cấp.
- Tích hợp thành công các tính năng hiện đại như Cloud Dashboard, cập nhật OTA, đồng bộ NTP và chế độ khẩn cấp bằng nút bấm vật lý.
- Hệ thống duy trì hoạt động ổn định nhờ cơ chế nguồn dự phòng thông minh.

Dù đã đạt được các yêu cầu về chức năng và phi chức năng, hệ thống vẫn còn một số hạn chế về sai số của cảm biến bán dẫn và cần tăng cường thêm lớp xác thực chứng chỉ CA cho kết nối MQTT/TLS.

3.2 Hướng phát triển

Để nâng tầm hệ thống lên quy mô ứng dụng thực tế chuyên sâu hơn, nhóm đề xuất các hướng phát triển:

- Nâng cấp độ chính xác: Thay thế cảm biến MQ bằng các dòng cảm biến kỹ thuật số (như NDIR hoặc Laser) để có số liệu chính xác tuyệt đối về bụi mịn và các loại khí đặc thù.
- Trí tuệ nhân tạo (AI): Triển khai mô-đun AI Summary để tóm tắt mức độ ô nhiễm theo giờ, cảnh báo xu hướng bất thường và đề xuất hành động an toàn.
- Bảo mật và quy mô: Tích hợp chứng chỉ CA đầy đủ cho MQTT/TLS trên ESP32 (thay cho `setInsecure()`), đồng thời mở rộng hệ thống thành mạng lưới nhiều node giám sát phân tán.
- Chuẩn hóa dữ liệu cảm biến: Bổ sung bước calibration để quy đổi từ giá trị ADC sang đơn vị ppm, tăng khả năng đối chiếu với tiêu chuẩn môi trường.

TÀI LIỆU THAM KHẢO

- [1] Rui Santos, “ESP32 Pinout Reference: Which GPIO pins should you use?,” Random Nerd Tutorials. Available: <https://randomnerdtutorials.com/esp32-pinout-reference-gpios/>
- [2] Components101, “MQ135 Gas Sensor for Air Quality,” 2020. Available: <https://components101.com/sensors/mq135-gas-sensor-for-air-quality>
- [3] HiveMQ, “MQTT Essentials - A Lightweight IoT Protocol.” Available: <https://www.hivemq.com/mqtt/>
- [4] Real Time Engineers Ltd., “FreeRTOS API Reference.” Available: <https://www.freertos.org/a00106.html>